UML, DFD and Project Diagrams

Notation, detailed explanation, drawing method and viva questions

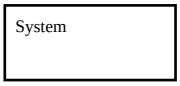
Worked example throughout: Blood Bank and Donor Management System (BBDMS)

1	Use Case Diagram
2	Activity Diagram
3	Data Flow Diagrams (Level 0 and Level 1)
4	Class Diagram
5	Sequence Diagram
6	Collaboration (Communication) Diagram
7	Work Breakdown Structure (WBS)
8	Gantt Chart / Project Schedule
9	Cross-diagram viva questions and quick comparison
10	Lab-day checklist

1. Use Case Diagram

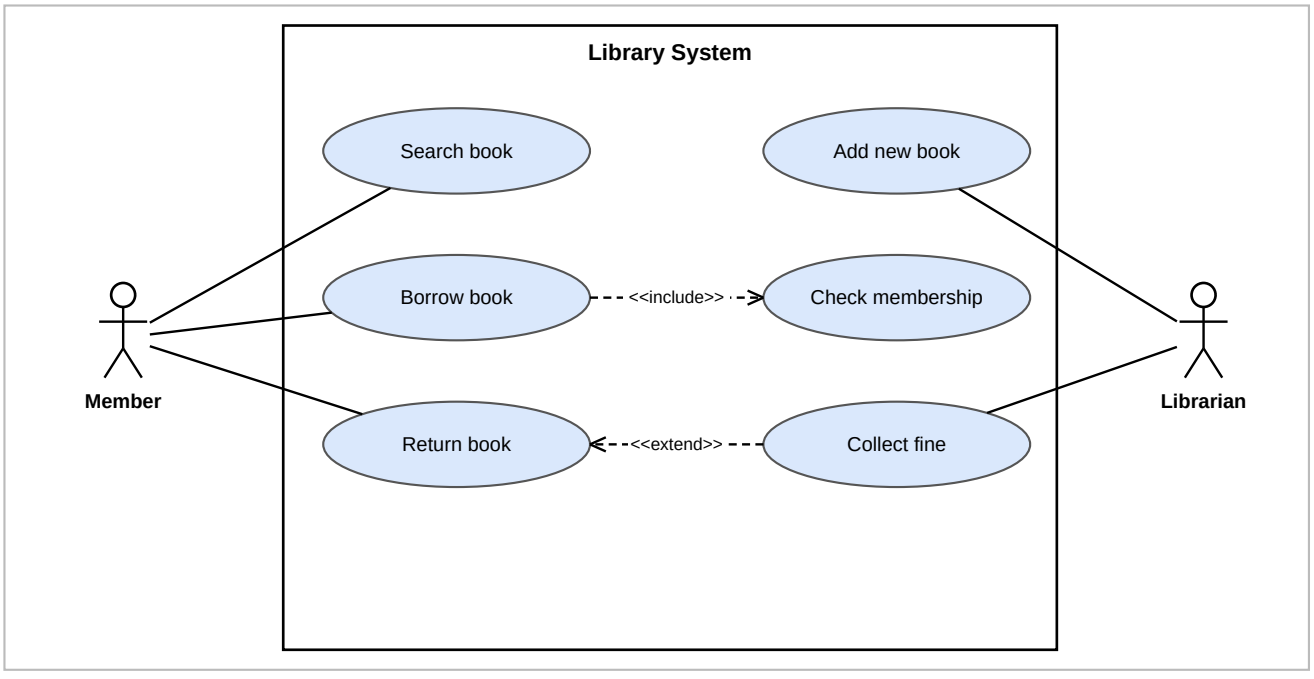
What it is. A use case diagram shows *what* a system does from the point of view of the people and external systems that use it. It does not show how the work is done internally. It is drawn during requirements analysis and is the first diagram you should draw, because every other diagram refers back to the actors and functions identified here.

Notation

Symbol	Name	Meaning / how to use
	**Actor**	A role (not a particular person) that interacts with the system: Donor, Blood Bank Staff, Administrator, Hospital/Requester, Visitor, SMS/Email Gateway. Drawn outside the system boundary. Primary actors (who start a use case) are usually on the left, external supporting systems on the right.
	**Use case**	One complete function that gives the actor a result of value. Name = verb + noun: Register donor, Schedule donation, Approve request.
	**System boundary**	Rectangle around all use cases, labelled with the system name. Actors are outside it, use cases inside it.
	Association	Plain solid line between an actor and a use case: the actor takes part in that use case. No arrow head is needed.
	<<include>>	Dashed arrow from the base use case to a use case that is **always** executed as part of it (mandatory). Example: Schedule donation includes Health rescreen.
	<<extend>>	Dashed arrow from the optional use case **to** the base use case it adds to. Used for optional or conditional behaviour. Example: Download certificate extends Donation history.
	Generalization	Solid line with hollow triangle pointing to the parent. A child actor or use case inherits the parent behaviour (e.g. Administrator is a kind of Staff).

Easy Example: Library System

Two actors, six use cases. Read the picture first, then the notes below it.



How to read it:

- **Member** and **Librarian** are actors (outside the box). The box is the system boundary.
- **Borrow book <<include>> Check membership:** you can never borrow without checking the membership, so the check is mandatory. The arrow points to the included use case.
- **Collect fine <<extend>> Return book:** a fine is collected only if the book is late, so it is optional. The arrow points to the base use case (Return book).
- Each plain line means: this actor takes part in this use case.

Detailed explanation (BBDMS)

The BBDMS use case diagram has five human actors. A **Visitor** can only register as a donor. A **Donor** manages their profile, schedules a donation, views donation history, takes a health re-screen, downloads a certificate and receives reminders. **Blood Bank Staff** record the outcome of a donation, manage inventory, check availability, approve or reject requests, search donors and export donor lists. The **Administrator** sets stock thresholds and handles escalations when there is no stock. The **Hospital/Requester** submits and tracks blood requests. Relationships such as Submit request including Check availability, or Escalate no-stock extending Approve request, are shown with include and extend arrows.

How to draw it, step by step

- List every user role and external system from the case study: these are the actors.
- For each actor write what they want from the system as a verb + noun phrase: these are the use cases.
- Draw the system boundary rectangle and place use cases inside it, actors outside it.
- Join each actor to its use cases with plain lines.
- Look for a step that several use cases share: make it an include. Look for optional or exceptional behaviour: make it an extend (arrow to the base).
- Check: no use case is without an actor, no actor is without a use case, and use case names do not describe screens or database operations.

Common mistakes

- Drawing the extend arrow in the wrong direction (it points to the base use case, include points to the included one).
- Showing the flow of steps, which belongs in an activity diagram.
- Putting actors inside the boundary, or treating a person (Ravi) as an actor instead of a role (Donor).

Viva Questions and Model Answers

Q1. What is a use case diagram and when is it drawn?

It is a behavioural UML diagram that shows the functions of the system as seen by its users. It is drawn in the requirements analysis phase after the actors and requirements are identified.

Q2. What is an actor? Can an actor be a system?

An actor is a role outside the system that interacts with it. Yes, an external system such as an SMS/Email gateway is an actor.

Q3. Difference between include and extend?

Include is mandatory: the base use case always calls the included one. Extend is optional or conditional: the extending use case adds behaviour to the base only in some situations. Include arrow points to the included use case, extend arrow points to the base.

Q4. What is the system boundary?

The rectangle that separates what is inside the system (use cases) from what is outside (actors).

Q5. Can two actors be connected to the same use case?

Yes. For example both Staff and Administrator may take part in Approve request.

Q6. What is generalization between actors?

A child actor inherits all use cases of the parent and may add its own. Example: Administrator generalizes Staff.

Q7. Does a use case diagram show order of steps?

No. The order is shown in an activity or sequence diagram. A use case diagram only shows which functions exist and who uses them.

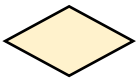
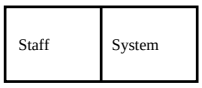
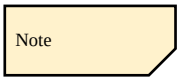
Q8. How are use cases named?

As a short verb + noun phrase from the actor's view, for example Record outcome, not Click the save button.

2. Activity Diagram

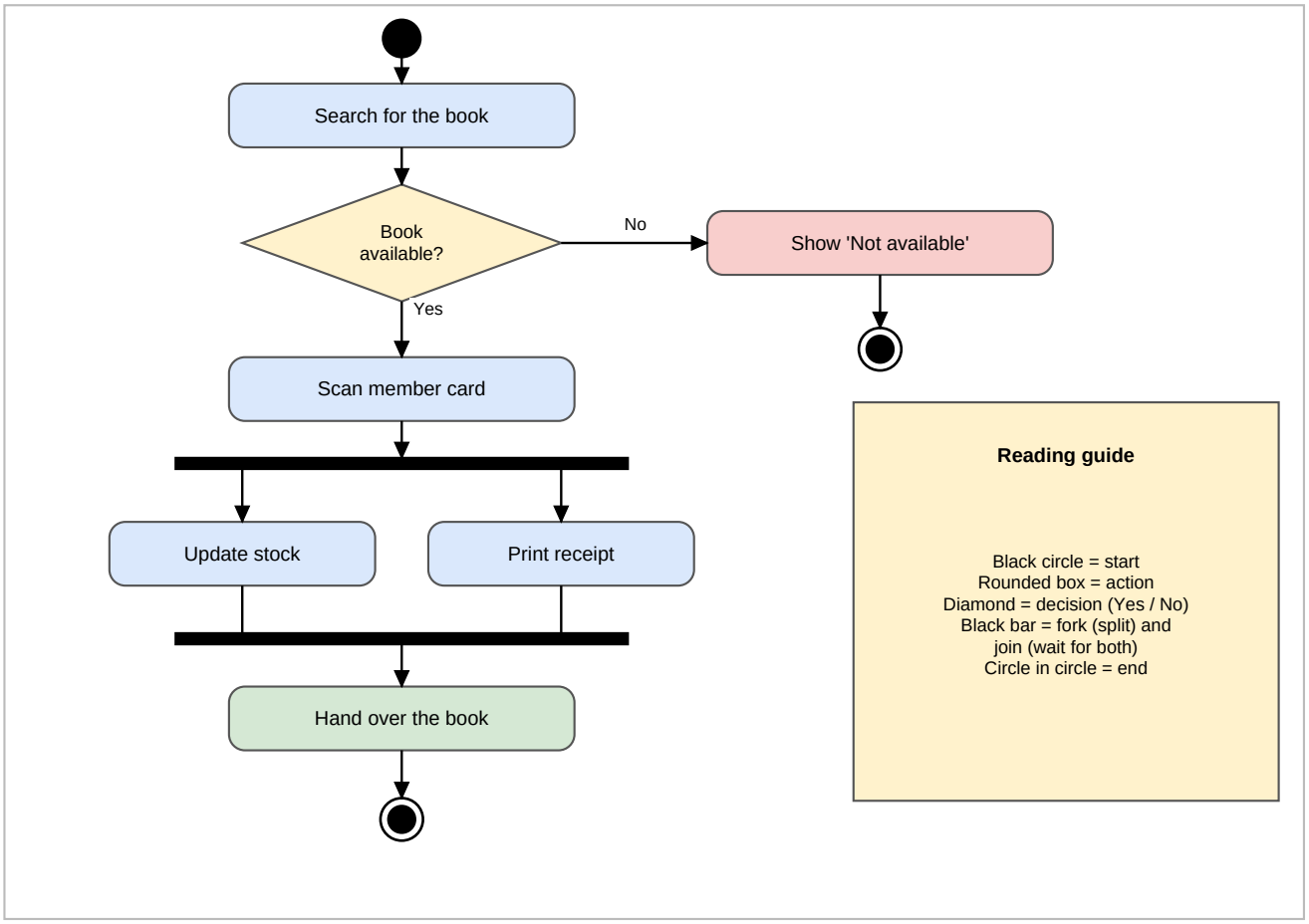
What it is. An activity diagram is a flowchart-like UML diagram that models the *workflow* of a use case or business process: the actions, the decisions, the parallel branches and the end states. It answers "in what order are things done and what happens in each case?"

Notation

Symbol	Name	Meaning / how to use
	**Initial node**	Filled black circle. Exactly one per diagram: where the flow begins.
	**Action / activity**	Rounded rectangle containing a verb phrase: Submit blood request, Record blood unit.
	Control flow	Solid arrow from one step to the next.
	**Decision / merge**	Diamond with one incoming flow and several outgoing flows. Each outgoing arrow carries a guard such as Yes, No, Full or partial. A diamond can also merge branches back together.
	Fork / join bar	Thick black bar. A fork splits one flow into parallel flows; a join waits for all of them before continuing. Example: Notify requester and Await response window run in parallel.
	**Final node**	Circle with a filled dot inside (bull's eye). Ends the whole flow. A diagram can have more than one.
	**Swimlane (optional)**	Vertical or horizontal partitions, one per actor/system, showing who performs each action.
	**Note**	Folded-corner box for comments, attached with a dashed line.

Easy Example: Borrowing a Book

One workflow from start to finish. Notice that every path of the decision ends in an End circle.



How to read it:

- The flow starts at the black circle, searches the book, then reaches a **decision**: Yes or No.
- **No** path: show a message and end. **Yes** path: scan the card, then **fork**.
- After the fork, *Update stock* and *Print receipt* happen at the same time (in parallel). The **join** bar waits until both are done.
- Last action: hand over the book, then End.

Detailed explanation (BBDMS: Blood Request Workflow)

The flow starts when the hospital submits a blood request with group, component, quantity and urgency. Staff review the request and check inventory. At the first decision, **Units available?**, there are two paths: if none are available the request is rejected, the requester is notified and the flow ends; if the units are available in full or in part, the request is approved and the units are reserved. After approval a fork starts two parallel actions: notify the requester of the approval, and await the requester's response window. A join waits for both. A second decision asks **Requester cancels?**: Yes releases the reserved units and ends; No issues the units and deducts them from inventory, then ends.

The donor workflow follows the same pattern: register and answer the health questionnaire, decision **Eligible?**, schedule appointment, fork (send confirmation, await date), join, attend, decision **Completed?**, record the unit as In Quarantine, screening tests, decision **Results satisfactory?**, then either discard the unit or add it to inventory as Available.

How to draw it, step by step

- Pick one use case or process and write its steps in order.
- Start with the initial node, then one rounded rectangle per action, joined with arrows.
- Whenever the process can go two ways, put a diamond and label every arrow leaving it.
- Steps that can happen at the same time go between a fork bar and a join bar.

- Every path must finish at a final node (or merge back into the main flow). Do not leave dangling arrows.
- Optional: colour the failure paths red and success paths green, and add swimlanes if different actors do different steps.

Common mistakes

- Decision diamonds without labels on the outgoing arrows.
- A fork with no matching join (parallel paths must be rejoined or each must end).
- Using it to show messages between objects (that is a sequence diagram).

Viva Questions and Model Answers

Q1. What does an activity diagram model?

The dynamic workflow of a process or use case: actions, decisions, concurrency and the order between them.

Q2. Difference between an activity diagram and a flowchart?

An activity diagram is a UML diagram that also supports parallel flows (fork/join), swimlanes and object flows. A basic flowchart has no concurrency notation.

Q3. What are fork and join?

A fork splits one flow into concurrent flows; a join synchronises them, so the next step starts only when all incoming flows have arrived.

Q4. Decision node versus merge node?

Both are diamonds. A decision has one input and many guarded outputs; a merge has many inputs and one output.

Q5. What is a swimlane?

A partition of the diagram that shows which actor, department or system is responsible for each action.

Q6. Can a diagram have more than one final node?

Yes, each alternative path (reject, cancel, success) may end at its own final node. It has only one initial node.

Q7. What is a guard condition?

A Boolean condition in square brackets or as a label on a decision's outgoing arrow, such as Yes, No, Full or partial. Only one guard should be true at a time.

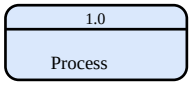
Q8. Which BBDMS step in your diagram runs in parallel and why?

Notifying the requester and awaiting the response window, because they do not depend on each other and both must be complete before the cancel decision.

3. Data Flow Diagrams (Level 0 and Level 1)

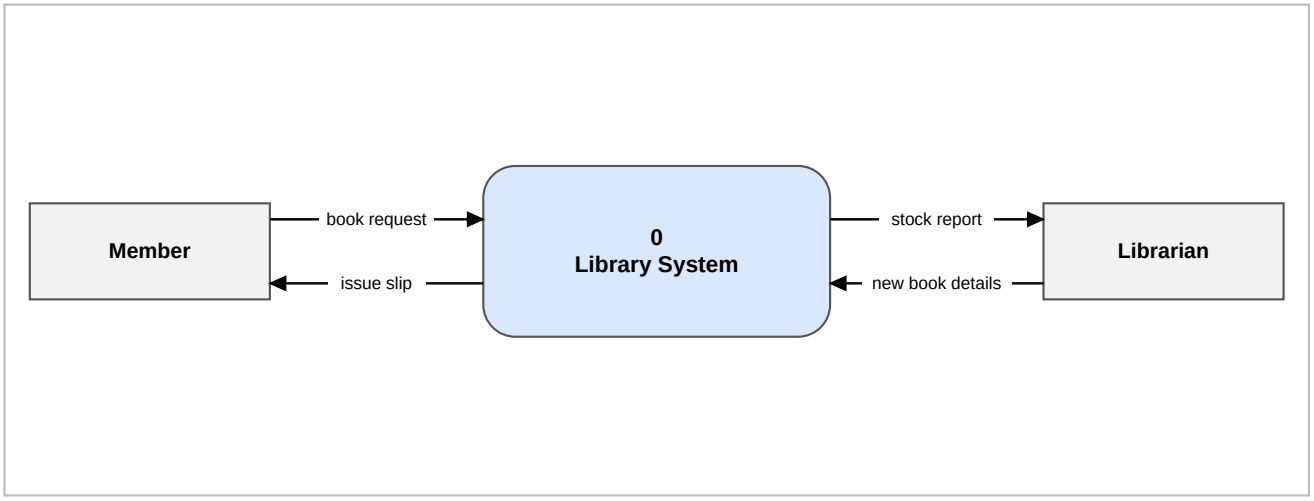
What it is. A DFD shows how *data* moves through a system: where it comes from, which processes transform it, where it is stored and where it goes. It is a structured-analysis tool and shows no timing, decisions or control. Diagrams are drawn in levels: Level 0 (context) first, then each process is broken down in Level 1, Level 2 and so on.

Notation (Yourdon / DeMarco style, as used in the BBDMS diagrams)

Symbol	Name	Meaning / how to use
	**External entity**	Rectangle. A person, organisation or system outside the system that is a source or destination of data: Donor, Hospital/Requester, SMS/Email Gateway.
	**Process**	Rounded rectangle with a number and a verb-phrase name: 1.0 Manage Donor, 4.0 Submit and Fulfill Request. It must have at least one input and one output.
	**Data store**	Open-ended rectangle labelled D1, D2...: Donor Records, Blood Inventory, Blood Requests. Data is kept here at rest.
	Data flow	Arrow labelled with the data that moves (a noun phrase): registration details, stock report, availability. Not an action.

Easy Example: Level 0 (context) DFD of the Library System

The whole system is ONE process in the middle. Only external entities around it, and no data stores.

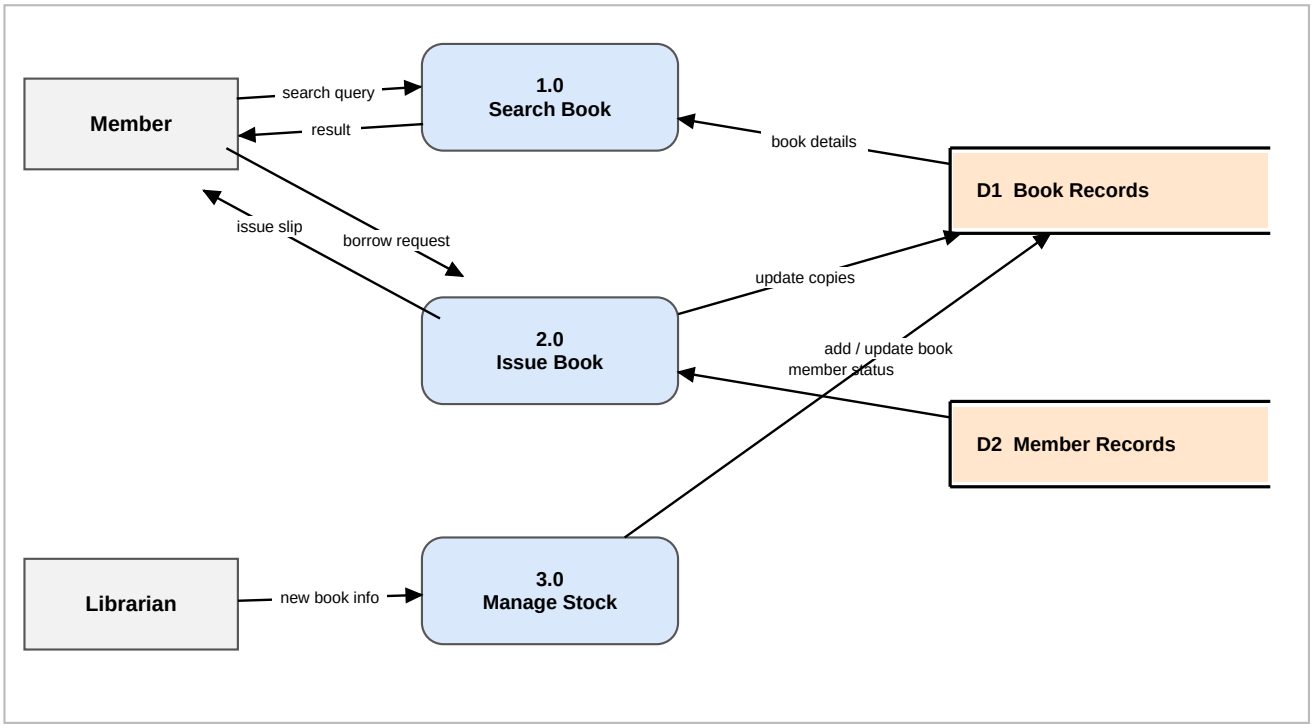


How to read it:

- Each arrow is **data** (a noun): *book request*, *issue slip*, *new book details*, *stock report*.
- Two arrows between the same pair show data going in and data coming back.

Easy Example: Level 1 DFD of the Library System (the same system opened up)

The single process 0 is split into three numbered processes and two data stores.



How to read it:

- Same entities as Level 0 (Member, Librarian): this is what **balancing** means.
- Process **2.0 Issue Book** reads D2 (is the member valid?) and updates D1 (one copy less), then gives the Member an issue slip.
- No arrow goes directly from an entity to a store: a process is always between them.

Level 0: Context diagram

The whole BBDMS is drawn as one process, numbered 0, in the centre. Around it are the five external entities: Donor, SMS/Email Gateway, Blood Bank Staff, Blood Bank Administrator and Hospital/Requester. Arrows show only the major data exchanged: registration and appointment confirmation with the donor, alerts and reminders to the SMS/Email gateway, inventory updates and donor lists with staff, thresholds and usage reports with the administrator, and blood requests and status with the hospital. There are **no data stores** at this level.

Level 1

The single process is split into five numbered processes: 1.0 Manage Donor, 2.0 Record Donation, 3.0 Manage Blood Inventory, 4.0 Submit and Fulfill Request, 5.0 Search Donors and Availability. Three data stores appear: D1 Donor Records, D2 Blood Inventory, D3 Blood Requests. For example, the donor registers in 1.0, an eligible donor goes to 2.0 which writes profile data to D1; the staff update stock into D2; the hospital's request enters 4.0 which reserves or deducts units in D2 and keeps the request record in D3; 5.0 reads availability from D2 and matching donors from D1.

Rules of DFDs

- **Balancing:** the inputs and outputs of Level 1 must match the flows of the Level 0 process they came from, with the same external entities.
- Data never flows directly between two entities, between two stores, or from an entity directly to a store: a process must always be in between.
- **No black hole** (a process with inputs but no outputs) and **no miracle** (outputs with no inputs).
- Processes are numbered; stores are D1, D2...; flows are labelled with nouns.
- No decisions, loops or time sequence: use an activity or sequence diagram for those.

How to draw it, step by step

- Draw Level 0 first: one process, all entities, and the main data between them.
- Decide the 3 to 6 main functions: they become 1.0, 2.0, 3.0 in Level 1.
- Add data stores wherever data must be kept between processes.
- Connect everything with labelled arrows following the rules above, then check balancing against Level 0.

Viva Questions and Model Answers

Q1. What is a DFD?

A diagram that shows how data flows between external entities, processes and data stores in a system.

Q2. What is a context diagram?

The Level 0 DFD, showing the whole system as one process with its external entities and data flows. It shows no data stores.

Q3. What are the four DFD symbols?

External entity, process, data store and data flow.

Q4. What is balancing?

The inputs and outputs of a lower-level DFD must equal those of the parent process, so that no data appears or disappears when a process is decomposed.

Q5. Difference between a data flow and a control flow?

A data flow carries data items between components. Control flow (used in flowcharts and activity diagrams) shows the order of execution. A DFD has no control flow.

Q6. Can data flow directly from an entity to a data store?

No. It must pass through a process, which validates and processes the data first.

Q7. What is a black hole and a miracle?

A black hole is a process that only receives data and produces no output. A miracle is a process that produces output without any input.

Q8. Difference between a DFD and a flowchart?

A DFD shows movement of data and has no decisions or sequence; a flowchart shows the sequence of steps and decisions in an algorithm.

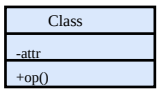
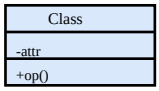
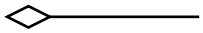
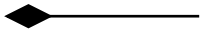
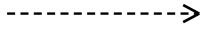
Q9. Why are process names verb phrases and flows noun phrases?

A process does something to data (Manage Donor), while a flow is the data itself (eligible donor, profile data).

4. Class Diagram

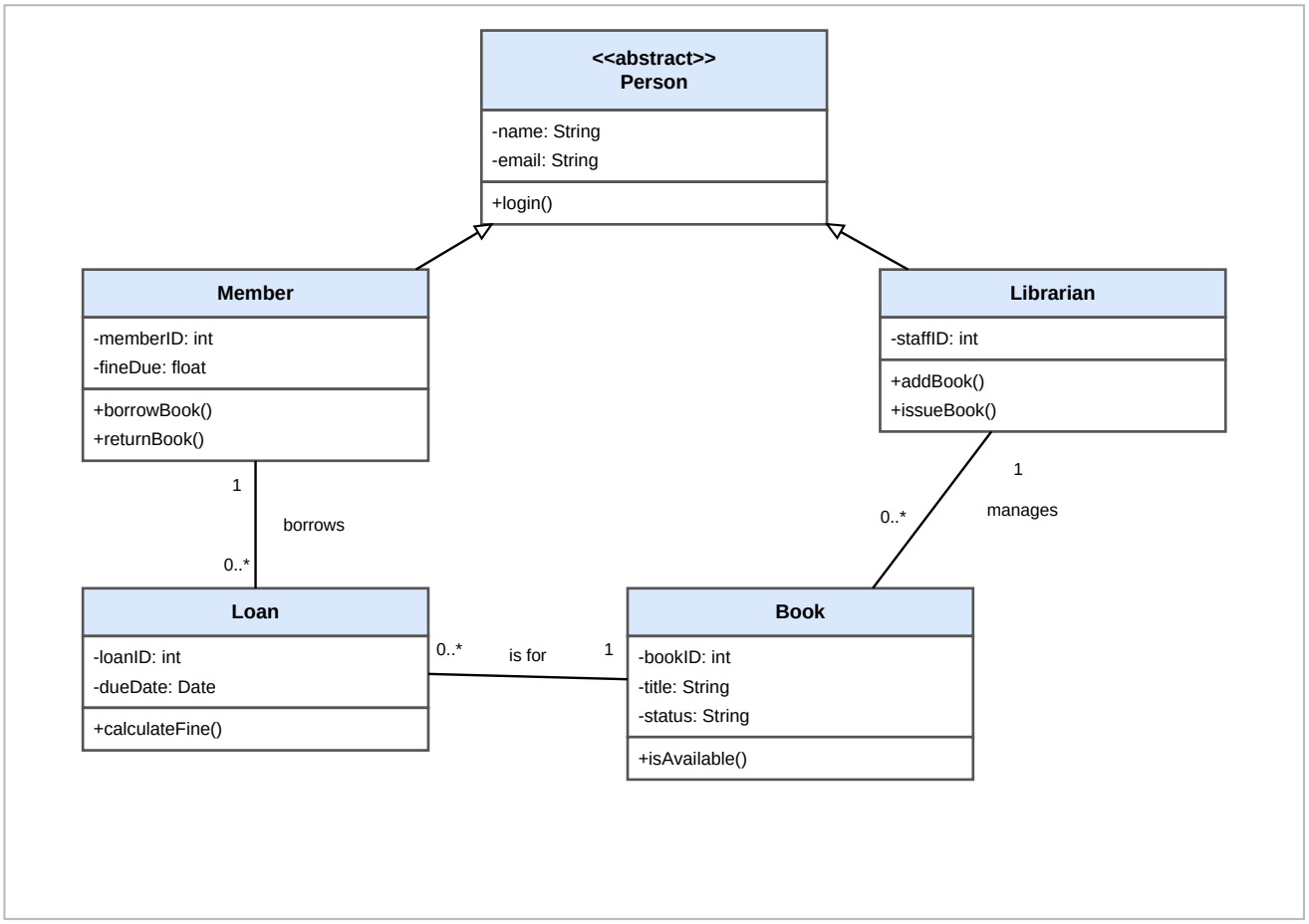
What it is. A class diagram is the main *static structure* diagram of UML. It shows the classes of the system, their attributes and operations, and the relationships between them. It is the blueprint for the database tables and for the code.

Notation

Symbol	Name	Meaning / how to use
	**Class**	Box with three compartments: name (top), attributes (middle), operations (bottom). Abstract classes are marked <<abstract>> or in italics.
	**Visibility**	**+** public, **-** private, # protected. Attribute format: -name: Type. Operation format: +login(): boolean.
	Generalization	Solid line with hollow triangle at the parent. Donor, BloodBankStaff, BloodBankAdministrator and Requester all inherit from the abstract class User. Read as "is-a".
	Association	Plain line between classes that know about each other, named with a verb (schedules, produces).
	**Multiplicity**	Numbers at the line ends: 1 exactly one, 0..1 optional, 0..* zero or more, 1..* one or more. One Donor schedules 0..* Appointments.
	**Aggregation**	Hollow diamond on the whole side: a weak "has-a" where the part can exist without the whole.
	**Composition**	Filled diamond on the whole side: a strong "owns-a"; the part dies with the whole.
	**Dependency**	Dashed arrow: one class uses another temporarily (for example NotificationService used by BloodRequest).

Easy Example: Library System classes

Each box has 3 parts: name, attributes (data), operations (actions).



How to read it:

- **Person** is abstract (we never create a plain Person). **Member** and **Librarian** inherit from it: hollow triangle arrow pointing to the parent, read as *is-a*.
- **Member 1 --- 0..* Loan:** one member can have many loans (or none); each loan belongs to exactly one member.
- **Loan 0..* --- 1 Book:** a book can be in many loans over time; each loan is for one book.
- **-** means private, **+** means public.

Detailed explanation (BBDMS)

User is an abstract class holding userID, email, passwordHash, phone and the operations login() and logout(). Four classes inherit it. **Donor** adds dateOfBirth, gender, bloodGroup and eligibilityStatus, with register(), updateProfile(), checkEligibility() and viewDonatedBloodDetails(). **BloodBankStaff** has recordDonationOutcome(), reviewRequest() and adjustInventory(). **BloodBankAdministrator** has setThreshold(), manageStaffAccounts(), generateReport() and approveEmergencyRequest(). **Requester** (a hospital) has submitRequest(), trackRequest() and cancelRequest().

The business classes form a chain: a Donor completes a **HealthScreening** and schedules an **Appointment**; an appointment produces a **Donation**; a donation yields a **BloodUnit**; blood units are stocked in **BloodInventory** (counts per blood group and component, with a threshold). A Requester raises a **BloodRequest**, which is resolved via **RequestFulfillment** that draws units from inventory. **NotificationService** sends messages (channel, message, sentDate) to recipients.

How to draw it, step by step

- Underline the nouns in the case study: candidates for classes. Drop duplicates and things that have no data or behaviour.
- For each class list its attributes (data) and operations (actions), with visibility and types.

- Find parent-child pairs and join them with generalization arrows (child to parent).
- Join the other related classes with named associations and add multiplicity at both ends.
- Check: every class has an identifier attribute, every line has multiplicity, no class is isolated.

Common mistakes

- Inheritance arrow pointing the wrong way (it points to the parent).
- Missing multiplicity, or the same multiplicity written on the wrong end of the line.
- Turning every noun into a class, including attributes such as 'name' or 'date'.

Viva Questions and Model Answers

Q1. What is a class diagram?

A UML structure diagram that shows the classes of a system, their attributes, operations and the relationships between them.

Q2. What do +, - and # mean?

Public, private and protected visibility respectively.

Q3. What is an abstract class? Give an example from BBDMS.

A class that cannot be instantiated and exists to be inherited. User is abstract; only Donor, Staff, Administrator or Requester objects are created.

Q4. Difference between association, aggregation and composition?

Association is a general link. Aggregation is a weak whole-part relation (hollow diamond) where parts live independently. Composition is a strong whole-part relation (filled diamond) where parts are destroyed with the whole.

Q5. What is multiplicity? Give an example.

The number of objects that can take part in a relationship. One Donor can have 0..* Appointments; each Appointment belongs to exactly 1 Donor.

Q6. What does the triangle arrow mean?

Generalization (inheritance): the child class inherits attributes and operations of the parent. Read as is-a.

Q7. How do you identify classes from a case study?

Pick the nouns in the requirements, keep those that have attributes and behaviour and are relevant to the system, discard synonyms and mere attributes.

Q8. What is the difference between a class and an object?

A class is the blueprint; an object is a specific instance with values, for example :Donor named Asha with bloodGroup O+.

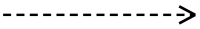
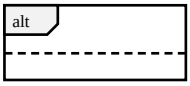
Q9. How is a class diagram converted to a database?

Each class becomes a table, attributes become columns, the identifier becomes the primary key, and associations become foreign keys (or link tables for many-to-many).

5. Sequence Diagram

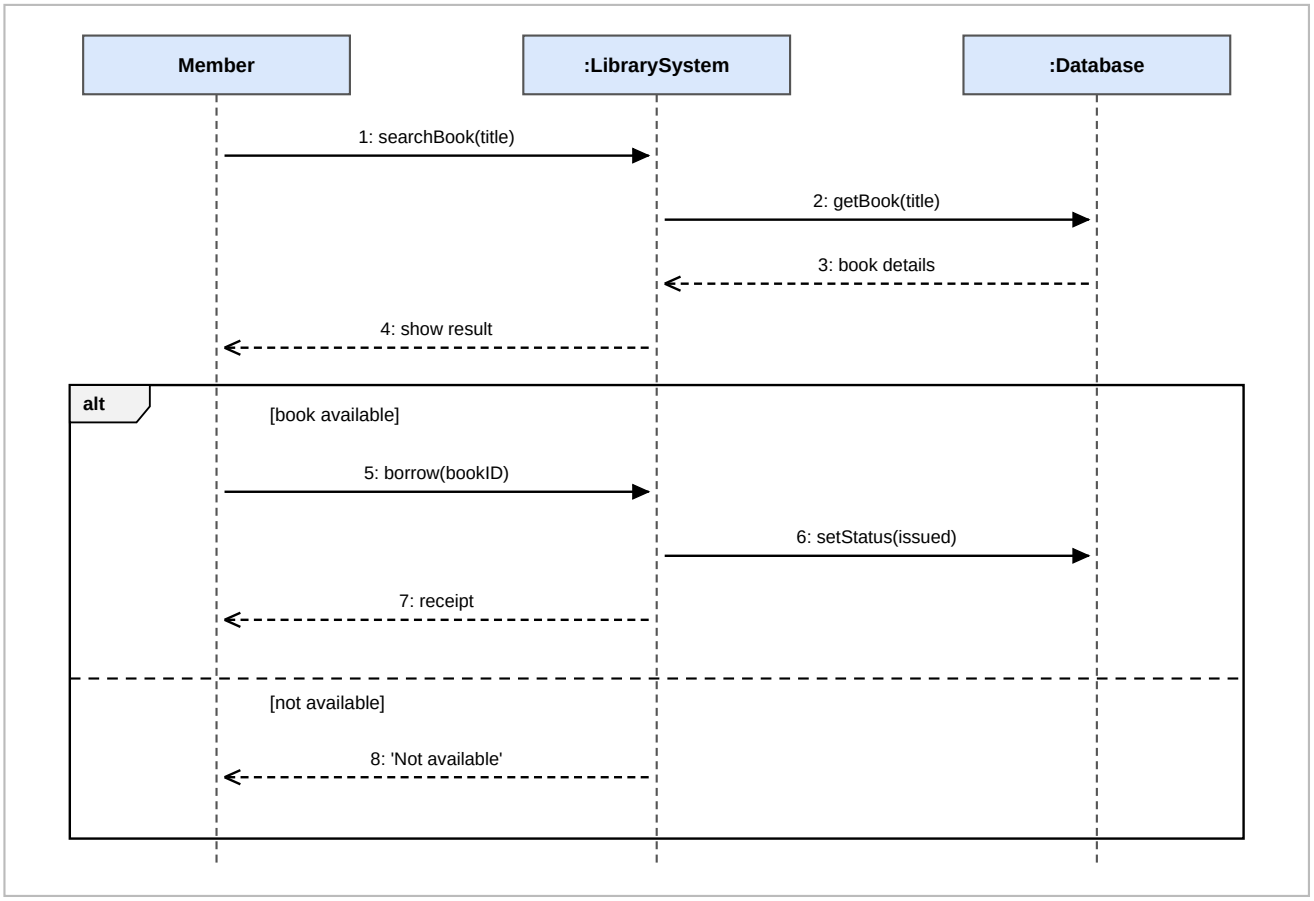
What it is. A sequence diagram shows how objects interact *in time order* to carry out one scenario. Objects are placed across the top, time runs down the page, and arrows between lifelines are the messages.

Notation

Symbol	Name	Meaning / how to use
	**Actor / object**	Participant at the top. Actors are stick figures, system parts are boxes. Order left to right by first involvement.
	**Lifeline and activation**	Dashed vertical line = the object's life. A thin white rectangle on it (activation bar) = the period it is busy processing.
	**Synchronous message**	Solid line with filled arrow head: a call, e.g. submitRequest(group, component, qty). The sender waits.
	**Return message**	Dashed line with open arrow head: the reply, e.g. acknowledgement.
	**Self message**	Arrow looping back to the same lifeline: the object does something internally, e.g. assign request no., status = Submitted.
	**Combined fragment**	Frame with a keyword: **alt** (alternatives, separated by a dashed line with a [guard] each), **opt** (optional), **loop** (repeat), **par** (parallel).

Easy Example: Search and Borrow a Book

Time runs from top to bottom. Objects are across the top.



How to read it:

- Solid arrow = call (message). Dashed arrow = return/reply.
- Messages 1 to 4: the member searches; the system asks the database and shows the result.
- The **alt** box has two alternatives with [guards]: if the book is available the borrow is recorded and a receipt is returned; otherwise only a 'Not available' message is shown. Only one part runs.

Detailed explanation (BBDMS: Blood Request Fulfillment)

The participants are Hospital/Requester, BBDMS, Blood Bank Staff, Administrator and Notification Service. The requester calls *submitRequest(group, component, qty, date, isEmergency)* on the BBDMS. The BBDMS assigns a request number, sets the status to Submitted and sends an acknowledgement through the Notification Service. Staff call *reviewRequest(requestID)* and the system shows the available quantity. An **alt** frame then models three outcomes:

- **[Sufficient stock]**: staff approve; the system sets status Approved and reserves the units; later issueUnits(requestID) deducts the units, sets status Fulfilled and the Notification Service informs the requester.
- **[Partial stock]**: staff approvePartial(requestID, availableQty); the unfulfilled remainder is recorded and a no-stock alert is escalated to the Administrator.
- **[No stock and Emergency]**: the system lists eligible due/overdue donors; if the request still cannot be met staff reject(requestID, reason) and the requester is notified of the rejection.

How to draw it, step by step

- Choose ONE scenario (one use case) and list the objects that take part.
- Place the actor first on the left, then the objects in the order they are first used.
- Draw lifelines downward and add messages top to bottom in time order, one arrow per row, each with a name and parameters. Number them 1, 2, 3.
- Use a dashed arrow for every return value.
- Wrap different outcomes in an alt frame with a [guard] condition for each part.

Common mistakes

- Showing a message to an object that is not on the diagram, or a diagonal arrow (keep arrows horizontal).
- Mixing several scenarios in one diagram without alt/opt frames.
- Message names that are not operations of the target class (they should match the class diagram).

Viva Questions and Model Answers

Q1. What does a sequence diagram show?

The interaction between objects for one scenario, arranged in time order.

Q2. What is a lifeline?

The vertical dashed line under an object representing its existence over time.

Q3. What is an activation bar?

A thin rectangle on a lifeline showing the time during which the object is executing an operation.

Q4. Difference between synchronous and asynchronous messages?

Synchronous (filled head): the sender waits for the reply. Asynchronous (open head): the sender continues without waiting.

Q5. What is the alt fragment?

A frame with two or more alternatives, each with a guard condition; exactly one is executed. It is the if-else of sequence diagrams.

Q6. What does a dashed arrow represent?

A return message (reply) to the caller.

Q7. How are sequence and class diagrams related?

Each message arrow corresponds to an operation of the receiving object's class, so the sequence diagram helps find missing operations in the class diagram.

Q8. Difference between a sequence and a collaboration diagram?

Both show the same interactions. Sequence emphasises time order (vertical axis); collaboration emphasises object relationships and uses numbering for order.

Q9. What is a self message?

A call from an object to itself, shown as an arrow that loops back to its own lifeline, e.g. assigning a request number.

6. Collaboration (Communication) Diagram

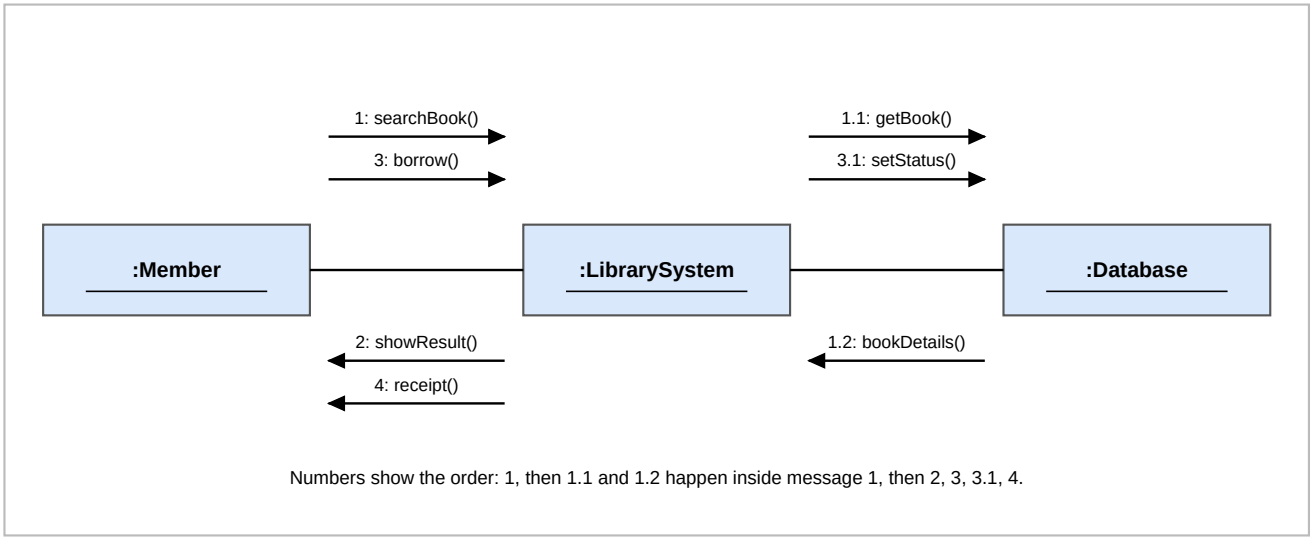
What it is. A collaboration diagram (called a communication diagram in UML 2) shows the same interactions as a sequence diagram but organised around the *links between objects*. Order is shown by **numbering** the messages instead of by position on the page.

Notation

Symbol	Name	Meaning / how to use
:Object	**Object**	Rectangle labelled :ClassName (or objectName:ClassName), underlined: :Donor, :Appointment, :BloodInventory.
	Link	A line between two objects that communicate with each other.
2: submit()	**Numbered message**	Arrow along a link with a sequence number and the operation: 1: scheduleAppointment(). Nested or follow-up calls use decimals: 8.1: statusUpdate().
	Return / reply	Shown as another numbered message in the opposite direction, e.g. 9.1: reduceInventory().

Easy Example: the same Search and Borrow scenario

A collaboration diagram shows the same messages as the sequence diagram, but laid out around the links between objects.



How to read it:

- Objects are named :ClassName; the plain line between two objects is a **link**.
- Message numbers give the order. **1.1** and **1.2** are nested inside message 1: while searching, the system calls the database and gets the details back.
- Compare with the sequence diagram above: same messages, different layout.

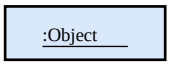
Detailed explanation (BBDMS)

Objects: :Donor, :Appointment, :Donation, :BloodUnit, :BloodInventory, :BloodBankStaff, :Administrator, :Requester, :BloodRequest, :ReqFulfillment, :NotificationSvc. **Donor workflow (1 to 4):** 1 scheduleAppointment() from Donor to Appointment; 2 completeDonation() to Donation; 3 createUnit() to BloodUnit; 4 addToInventory() into BloodInventory. **Requester workflow (5 to 11):** 5 monitorStock() by staff on inventory; 6 submitRequest() from Requester to BloodRequest; 7 checkAvailability(); 8 reviewAndApprove() by staff with 8.1 statusUpdate() back to the request; 9 issueUnits() to ReqFulfillment with 9.1 reduceInventory(); 10 manageInventory() by the Administrator; 11 notifyRequester() through the NotificationService with 11.1 notificationLog().

How to draw it, step by step

- Take the same scenario as the sequence diagram and list the objects.
- Draw the objects as boxes in any convenient layout, placing objects that talk to each other close together.
- Join communicating objects with link lines.
- Draw an arrow on each link for every message and number them in time order (1, 2, 2.1, 3...). Nested calls get decimals.
- You may convert it from the sequence diagram: every sequence arrow becomes a numbered arrow on the link.

Sequence versus collaboration at a glance

Symbol	Name	Meaning / how to use
	**Sequence**	Time is the vertical axis; easy to read the order; lifelines and activation bars; supports alt/opt/loop frames.
	**Collaboration**	Layout is free; shows which objects are connected; order given by numbers; compact for few objects.

Viva Questions and Model Answers

Q1. What is a collaboration diagram?

A UML interaction diagram that shows objects, the links between them and the numbered messages exchanged.

Q2. How is the order of messages shown?

By sequence numbers on the messages: 1, 2, 3, with decimals such as 2.1 for nested calls.

Q3. What is the notation for an object?

A rectangle with an underlined label :ClassName or name:ClassName.

Q4. Are sequence and collaboration diagrams equivalent?

They show the same information and can be converted into each other; they differ in emphasis (time versus structure).

Q5. When would you prefer a collaboration diagram?

When you want to show how objects are linked and when there are few objects with many links; a sequence diagram is better for long time-ordered scenarios.

Q6. What does 8.1 mean in the BBDMS diagram?

A message nested within message 8: while BloodBankStaff performs reviewAndApprove(), the BloodRequest status update 8.1 is sent back.

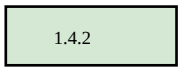
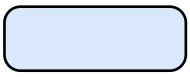
Q7. What is a link?

An instance of an association: a connection between two objects along which messages can be sent.

7. Work Breakdown Structure (WBS)

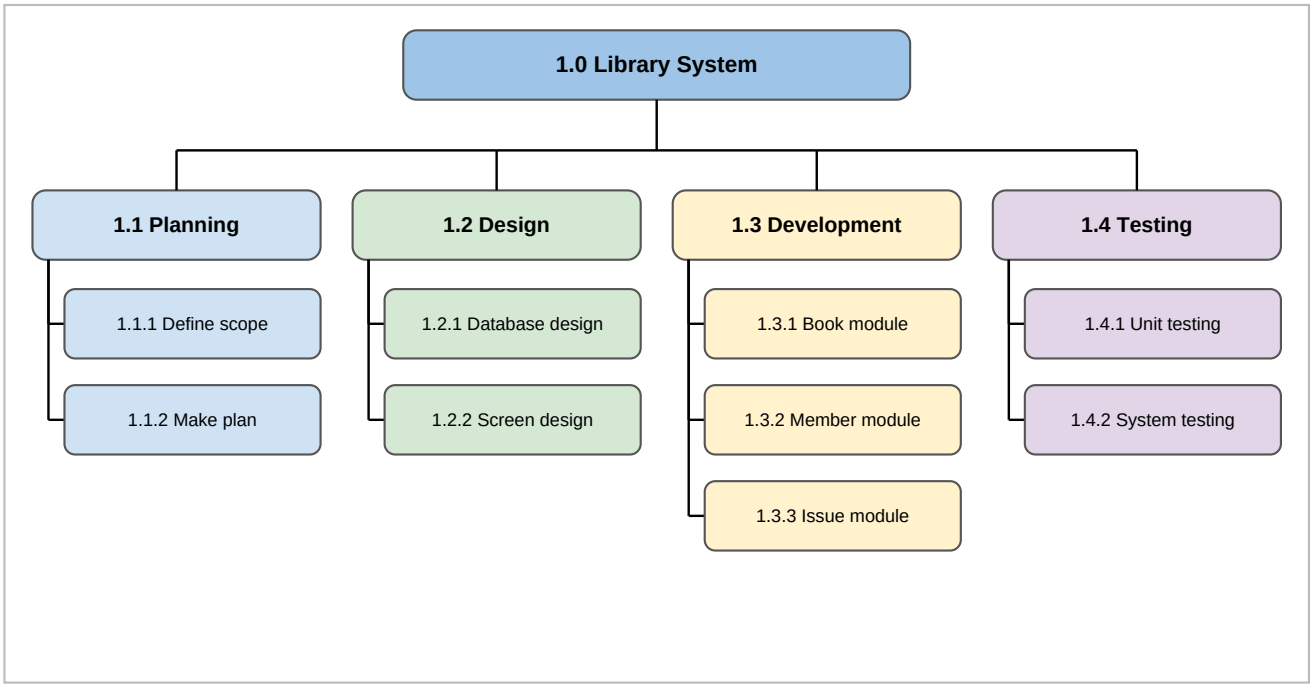
What it is. A WBS is a hierarchical, deliverable-oriented decomposition of the total work of the project into smaller and smaller pieces. It tells you *what* must be done, and it is the base for estimation, scheduling, costing and assigning responsibility.

Notation

Symbol	Name	Meaning / how to use
	**WBS box**	Each box has a number and a name. Level 1 = the project (1.0), Level 2 = major phases (1.1 to 1.7), Level 3 = work packages (1.4.1, 1.4.2...).
	**Hierarchy line**	Orthogonal line from a parent down to its children. It shows decomposition only, not sequence or dependency.
	**Colour coding**	One colour per phase, used for the phase and all its children, to make branches easy to read.

Easy Example: Library System WBS

Level 1 is the project, Level 2 the phases, Level 3 the work packages. Same colour = same branch.



How to read it:

- Numbering shows the parent: **1.3.2** belongs to **1.3**, which belongs to **1.0**.
- **100% rule:** 1.3.1 + 1.3.2 + 1.3.3 together make up all of 'Development'.
- There is no time or order here, only *what* has to be done. The Gantt chart adds the *when*.

Detailed explanation (BBDMS)

Level 1 is **1.0 Blood Bank and Donor Management System**. Level 2 has seven phases: 1.1 Project Planning, 1.2 Requirements Analysis, 1.3 System Design, 1.4 System Development, 1.5 Testing, 1.6 Deployment and 1.7 Maintenance. Level 3 work packages include, for example: under Planning, Define Scope, Identify Stakeholders, Feasibility Study and Project Plan and Timeline; under Development, Donor Registration and Profile Management, Donation Scheduling and History, Blood Inventory Management, Emergency Blood Request Module, Donor and Availability Search, Notification System (e-mail/SMS), Admin Panel and Reports, and Authentication and Security; under Testing, Unit, Integration, System and User Acceptance Testing; under Deployment, Server Configuration (Ubuntu/Nginx), Database Deployment (PostgreSQL), Application Deployment and Staff User Training.

Rules for a good WBS

- **100% rule:** the children together must cover all of the parent's work, nothing more and nothing less.
- **Mutually exclusive:** no overlap between sibling boxes.
- Focus on **deliverables / outcomes** (nouns), not on a sequence of steps.
- Go down until a work package can be estimated, assigned to one owner and tracked (roughly 8 to 80 hours, the 8/80 rule).
- Numbering must follow the hierarchy: 1.4 is a child of 1.0, 1.4.2 is a child of 1.4.

How to draw it, step by step

- Write the project name at the top as 1.0.
- List the major phases or deliverables below it as Level 2 and number them.
- Split each phase into work packages (Level 3) and number them.
- Colour each branch, align boxes in rows per level and connect with orthogonal lines.
- Check the 100% rule and that every work package appears again as an activity in the Gantt chart.

Viva Questions and Model Answers

Q1. What is a WBS?

A hierarchical decomposition of the project scope into phases and work packages.

Q2. What is a work package?

The lowest level of the WBS: a small deliverable that can be estimated, scheduled and assigned to an owner.

Q3. What is the 100% rule?

The WBS must include 100% of the work defined by the project scope, and the child elements of each parent must add up to 100% of the parent.

Q4. Does a WBS show the order of tasks?

No. It shows decomposition only. The order and dependencies are shown in the Gantt chart or network diagram.

Q5. How is the WBS numbered?

By level: 1.0 project, 1.1 phase, 1.1.1 work package.

Q6. How are WBS and Gantt chart related?

Each lowest-level WBS work package becomes an activity in the Gantt chart with a duration, predecessors and dates.

Q7. Why is the WBS deliverable-oriented?

So that the project is divided by what is produced rather than by who does it or when, which makes scope control and progress measurement easier.

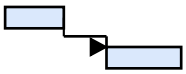
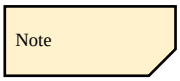
Q8. What are the 7 phases of the BBDMS WBS?

Project Planning, Requirements Analysis, System Design, System Development, Testing, Deployment and Maintenance.

8. Gantt Chart / Project Schedule

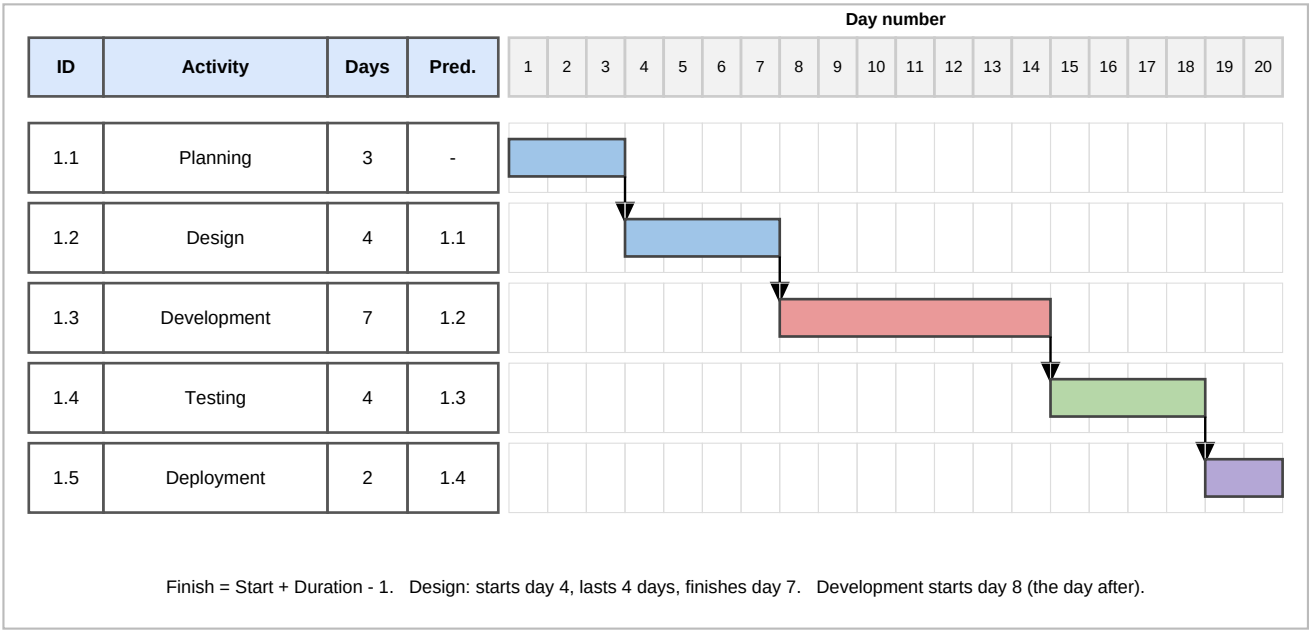
What it is. A Gantt chart is a horizontal bar chart of the project schedule. Each activity has a bar placed on a timeline; its length is the duration. It shows *when* things happen, which activities overlap and which depend on others.

Notation

Symbol	Name	Meaning / how to use
	**Task bar**	Horizontal bar: starts at the activity's start date, ends at its finish date. Colour shows the phase: planning/analysis, development, testing, deployment, maintenance.
	**Dependency arrow**	Arrow from a predecessor to its successor. Finish-to-Start (FS) is the most common: the successor starts after the predecessor finishes. Others: SS, FF, SF.
	**Milestone**	Diamond, duration 0: a significant point such as Project Start (activity 1.0).
	**Activity table**	Left-hand columns: ID, Activity, Duration (days), Predecessors, Start, Finish.

Easy Example: a 20-day Library System schedule

Each row is a WBS work package. A bar starts at its start day and its length is its duration.



Finish = Start + Duration - 1. Design: starts day 4, lasts 4 days, finishes day 7. Development starts day 8 (the day after).

How to read it:

- **Predecessor 1.1** means Design can start only after Planning finishes. The arrows show these links (Finish-to-Start).
- Here every activity depends on the previous one, so the whole chain is the **critical path**: any delay moves the final deadline.
- In a bigger project (like BBDMS) several activities start on the same day and run in parallel.

Detailed explanation (BBDMS)

The schedule runs from **17 Aug 2026 to 31 Oct 2026**. Project Planning (1.1, 5 days) follows the start milestone, then Requirements Analysis (7 days) and System Design (7 days) run one after another. Development begins on 06 Sep 2026. Activities 1.4.1 Donor Registration, 1.4.2 Donation Scheduling and 1.4.3 Blood Inventory Setup all depend on System Design (1.3) and therefore start together. 1.4.4 Blood Unit Testing and Quarantine Flow depends on 1.4.3, 1.4.5 Emergency Blood Request Module on 1.4.4, and then 1.4.6 Request Approval, 1.4.7 Donor and Inventory Search and 1.4.8 Notifications and Reports all depend on 1.4.5 and run in parallel. Unit testing (1.5.1) needs 1.4.6, 1.4.7 and 1.4.8 to be done, followed by Integration, System and User Acceptance Testing. Server configuration and database deployment run in parallel after UAT, then Application Deployment, User Training, and finally Bug Fixing, Performance Monitoring and Feature Enhancement.

Check before submitting: your PDF text says the project runs for 77 days, but the chart's footer says 76 days. Counting 17 Aug to 31 Oct inclusive gives 76 days (15 in August + 30 in September + 31 in October), so change the text to 76 days so both match.

Key calculations

- **Finish = Start + Duration - 1** (counting calendar days inclusively). Example: 1.1 Planning starts 18 Aug, 5 days, finishes 22 Aug.
- With a Finish-to-Start dependency, **Successor start = Predecessor finish + 1 day**. When there are several predecessors, use the latest finish.
- **Critical path** = the longest chain of dependent activities; a delay in any of them delays the whole project. Here it is the chain from planning through 1.4.3, 1.4.4, 1.4.5, testing and deployment to the last maintenance task.

How to draw it, step by step

- Take the work packages from the WBS as the activity list and give each an ID.
- Estimate a duration for each and decide predecessors.
- Compute start and finish dates for each activity from the dependencies.
- Draw the timeline (weeks or months), place one bar per activity from start to finish, colour by phase.
- Draw dependency arrows, mark milestones with diamonds, and add a legend and the total duration.

Viva Questions and Model Answers

Q1. What is a Gantt chart?

A bar chart of the project schedule with activities on one axis and time on the other, showing durations, overlaps and dependencies.

Q2. What is a milestone?

A zero-duration event marking an important point, such as project start or release, drawn as a diamond.

Q3. What is a predecessor?

An activity that must be done (or started) before another activity can begin.

Q4. What are the four dependency types?

Finish-to-Start, Start-to-Start, Finish-to-Finish and Start-to-Finish. Finish-to-Start is the most common.

Q5. What is the critical path?

The longest sequence of dependent activities, which determines the minimum project duration. Activities on it have zero slack.

Q6. What is slack (float)?

The time an activity can be delayed without delaying the next activity or the project finish.

Q7. How do you compute the finish date?

Finish = Start + Duration - 1 when counting inclusive calendar days.

Q8. Which activities in BBDMS run in parallel?

1.4.1, 1.4.2 and 1.4.3 (all after design); 1.4.6, 1.4.7 and 1.4.8 (all after 1.4.5); and 1.6.1 Server Configuration with 1.6.2 Database Deployment.

Q9. Difference between WBS and Gantt chart?

WBS shows what work there is (hierarchy, no time). Gantt chart shows when it is done (timeline, durations and dependencies).

Q10. Why is testing scheduled after all development modules?

Because the unit testing activity 1.5.1 has predecessors 1.4.6, 1.4.7 and 1.4.8: testing starts only when all of them are finished.

9. Cross-diagram Viva Questions and Quick Comparison

Which diagram answers which question?

Diagram	Type	Answers the question	Phase
Use case	Behavioural (UML)	Who uses the system and what for?	Requirements
Activity	Behavioural (UML)	In what order and with which decisions does the work flow?	Requirements / design
DFD	Structured analysis	Where does data come from, how is it processed, where is it stored?	Analysis
Class	Structural (UML)	What are the classes, data and relationships?	Design
Sequence	Interaction (UML)	Which messages are exchanged, in what time order?	Design
Collaboration	Interaction (UML)	Which objects are linked and what order are messages in?	Design
WBS	Project management	What work must be done (hierarchy)?	Planning
Gantt	Project management	When is each activity done and what depends on what?	Planning

Viva Questions and Model Answers

Q1. What is UML? Name its two groups of diagrams.

Unified Modeling Language, a standard visual language for modelling software. Diagrams are structural (class, object, component, deployment) and behavioural (use case, activity, state, sequence, communication).

Q2. In which order do you draw the diagrams in a project?

Requirements: use case and activity; analysis: DFD; design: class, then sequence and collaboration; planning: WBS and Gantt.

Q3. What is the difference between DFD and UML diagrams?

A DFD is a structured (function-oriented) analysis model showing data movement. UML is object-oriented modelling covering structure and behaviour.

Q4. Which diagrams use the same names and why?

Actors in use cases become entities in DFDs and participants in sequence diagrams; use cases become processes in DFDs; nouns become classes; operations in classes match messages in sequence diagrams. Consistent names make the model traceable.

Q5. What is static and dynamic modelling?

Static modelling shows structure that does not change with time (class diagram). Dynamic modelling shows behaviour over time (activity, sequence, collaboration).

Q6. What is a SRS and how are these diagrams linked to it?

The Software Requirements Specification lists the functional and non-functional requirements. The use case, activity, DFD, class and sequence diagrams are derived from it, as your BBDMS DFD document states.

Q7. Why did you choose draw.io?

It is free, works offline and in the browser, has UML and flowchart shape libraries and exports to PDF and images.

Q8. How do you check that your diagrams are consistent?

Check that actors and use case names match across diagrams, Level 1 DFD balances with Level 0, sequence messages exist as class operations, and every WBS work package appears in the Gantt chart.

10. Lab-day Checklist

- Read the case study twice. Underline **actors** (people/systems), **verbs** (use cases and activities), **nouns** (classes and data stores).
- Write the five lists on paper: actors, use cases, classes with attributes, data stores, the main workflow with decisions.
- Draw in this order: Use case, Class, Activity, Sequence, Collaboration, DFD 0, DFD 1, WBS, Gantt.
- Use the same names in every diagram. Add a title to each diagram and a 3 to 4 line description under it.
- Label every arrow: guards on decisions, data on DFD flows, messages on sequence arrows, multiplicity on class associations.
- Align shapes (Arrange menu), keep arrows straight and avoid crossings, and use colour sparingly.
- Save the .drawio file often, then export as PDF (File > Export as > PDF) at the end.
- Revise the viva questions of the diagrams you drew: examiners usually ask about notation first, then about your own diagram.